

Activity 4: Model-Driven Development (MDD) for a Banking System

1) Explain how MDD is a software development banking system.

* MDD is a software development approach where software models are created first, and application code is generated automatically from those models.

For a banking system:

* Create UML models such as class diagrams and use case diagrams

* Define customers, accounts, transactions and loans in the models

* Transform the models into source code automatically.

* Developers add only business specific logic

* This reduces coding errors and development time.

2) Identify the Platform-Independent Model and Platform specific Model PSM

PIM

* Describes system functionality without technology details.

* UML Class Diagram

* Use Case Diagram

* Banking Business rules

* Customer, Account, Transaction.

PSM

* Describes implementation using a specific platform

* Java classes.

* MySQL Database Tables

3) Advantages of generating code from models

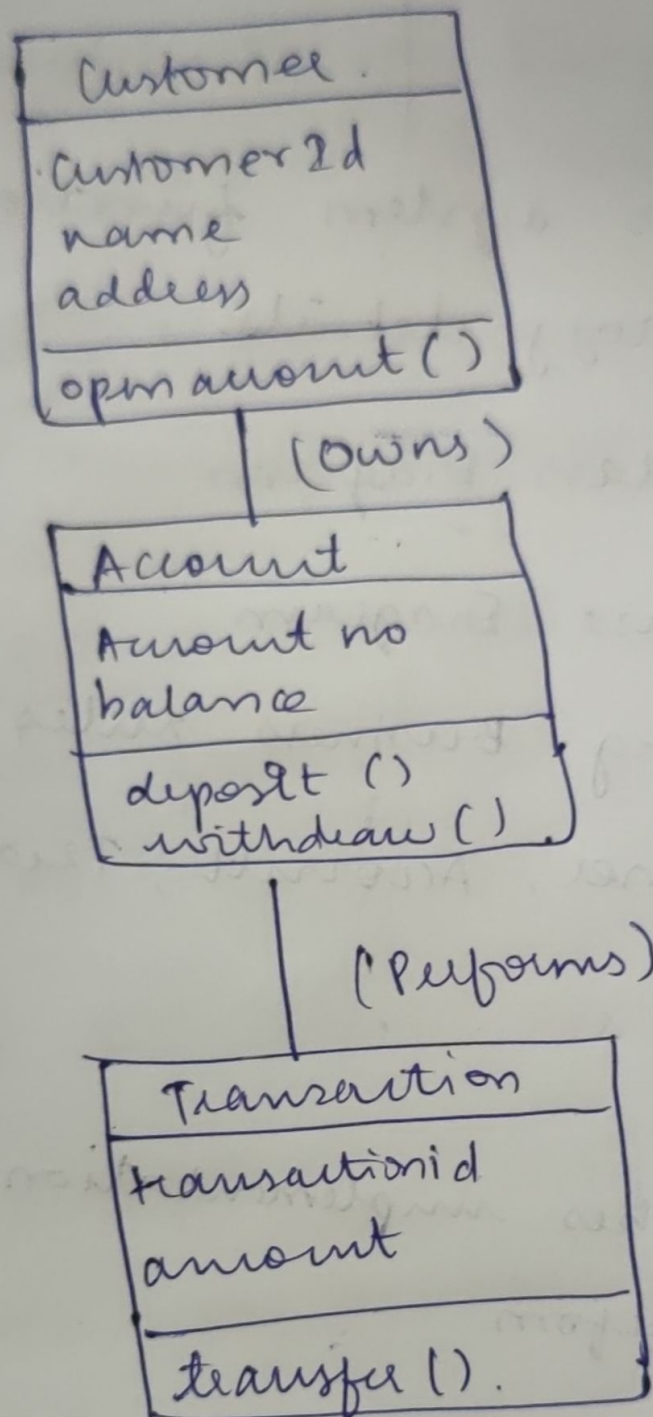
* Reduces coding errors

* Faster software development

* Improves software quality.

* Easier maintenance.

4) UML Class Diagram.



5) Limitations of using MDD

- * Requires skilled model designers
- * Initial Setup cost is high
- * Generated code may require manual modification.
- * Tool compatibility issues.

Activity 5: Agile Object Oriented Development for an E-learning Platform

1) Explain how Agile practices can be integrated with OOD

- * Developing software in short sprints.

- * Creating reusable classes and objects

- * Continuous testing and integration with frequent customer feedback

- * Delivering working software after each sprint.

2) Divide the project into suitable Agile iterations.

Sprint 1 : User registration, Login, User Profile

Sprint 2 : Online classes and Course management

Sprint 3 : Assignments and Quiz module

Sprint 4 : Discussion Forum and Progress Tracking

3) Identify the objects, classes, and relationships

Objects / classes:

- * Student
- * Teacher
- * Course
- * Quiz
- * Assignment
- * Forum
- * Progress

4) Explain how continuous feedback

improves software quality

- * Detects defects early
- * Improves user satisfaction
- * Reduces development risk.
- * Improves software quality through regular testing.